

Software Engineering Management and Development

Coursework : Swimming Pool Membership

Diya Panchal: M01025188

Ameya Pratap Singh Jamwal: M01034537

Clive Fernandes: M00991801

Jamaal Hasan: M00948164

Amarpreet Singh: M01048728

Supervised by Dr. Ahmed Eissa

Ahmed Bezan

Raheb Al-Mouaamen

Runem El-huni



**A THESIS SUBMITTED IN FULFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE BSC COMPUTER SCIENCE**

**SCHOOL OF COMPUTING AND INFORMATION SCIENCE
MIDDLESEX UNIVERSITY**

19/04/26

Table of Contents

1	Introduction	3
2	Methodology	4
2.1	System Architecture Overview	4
2.2	Rationale For Selecting Specific Data Structures	5
3	Implementation and Analysis	7
3.1	Operations with the Custom Binary Search Tree	7
3.2	Secure Authentication System	9
4	Results and Discussion	11
4.1	Testing Approach	11
4.2	Table of Test Cases	11
4.3	Technical Audit Analysis	13
5	Conclusion	14
6	Recommendations / Future Work	15

Chapter 1

Introduction

Managing a busy public swimming pool includes many complex operational and administrative aspects in addition to the normal day-to-day maintenance of the facility. It is essential to quickly process all of the membership subscription requests and user information along with securely authenticate all members/visitors on a daily basis. The traditional ways of handling these types of issues using paper based methods or even simple spreadsheets can lead to time consuming, error prone, and difficult to expand upon.

AquaSphere was created to provide an alternative method through which the problems mentioned above would be addressed. This will be accomplished by developing a comprehensive, web-based swimming pool membership management platform. As it currently exists, AquaSphere has replaced manually registering users at each individual pool location with a single point of entry that provides an easy way to register for pools and manage their own swimmer information. Additionally, AquaSphere is able to efficiently handle all of the member related functions, securely store and manage the user's personal information, and also allow swimmers to self-register. Although the fundamental back-end design and database schema have both been designed to support a multi-tiered Role-Based Access Control (RBAC) structure, the current front-end interface is limited to swimmers.

The main goal of this project is to create a scalable and secure system that will increase operational efficiencies while maintaining the integrity of sensitive authentication data. Through the emphasis on algorithmic optimizations and data security, AquaSphere provides a stable digital infrastructure ready to serve as a full-fledged facility management tool.

Chapter 2

Methodology

The AquaSphere uses multi-layered design as a means to isolate the user interface, application logic, and data management layers of its system architecture. Isolation of these different layers creates opportunities for improved system maintainability, scalability, and security. Relational databases will be used to manage the long term storage of data. The use of relational databases in managing data does not provide the best possible runtime performance when performing frequent operations. Therefore, it is necessary to carefully select the appropriate data structures for the application layer.

2.1 System Architecture Overview

The system has been segmented into three main layers that function independently.

- **Presentation Layer:** Manages user input via a web-based interface. At present, the web interface provides access to only swimmer (member)-related functionality; however, the architectural framework was designed to allow for development of additional staff and administrative interfaces in the future.
- **Application Layer:** Contains all of the system's business logic. Included in the business logic are processes for authentication via encryption algorithms, routing of members, and other processes related to temporary storage of member data in memory.
- **Data Layer:** Provides permanent storage of data. It utilizes a relational database management system that stores data in a very normalized format utilizing a SQLite relational database management system.

2.2 Rationale For Selecting Specific Data Structures

An important operational capability required for AquaSphere is the ability to retrieve the records of individual members nearly instantaneously by reception personnel. Although it would be technically feasible to utilize linear data structures such as arrays or linked lists to store member profiles, retrieving information stored in linear data structures typically involves sequentially traversing the structure, which results in $O(n)$ time complexity.

In order to enhance the efficiency of the search process utilized by reception personnel, the back-end of AquaSphere utilizes a binary search tree (BST) as an index based upon an algorithm. As described above, a BST allows for efficient searching, inserting, and deleting of data elements within the tree while maintaining an average-case time complexity of $O(\log n)$ due to the fact that it takes advantage of the property of having two child nodes (i.e., left child node and right child node), one with values less than the parent node value and another with values greater than the parent node value. Furthermore, since the members' email addresses are used as the sorting keys for their respective entries in the BST, the search space for each step of the algorithm will be significantly reduced due to the lexicographic ordering nature of the BST (i.e., alphabetical ordering).

As depicted in Figure 1, a simple example illustrates how a staff member could

```
graph TD
    A["Root Node: m@email.com"] --> B["Left Child: a@email.com"]
    A --> C["Right Child: z@email.com"]
    B --> D["Left Child: NULL"]
    B --> E["Right Child: c@email.com"]
    C --> F["Left Child: p@email.com"]
    C --> G["Right Child: NULL"]
```

Figure 2.1: Illustration of a simplified lexicographically ordered binary search tree (Reference: Cormen et al., 2009)

search for c@email.com. First, the algorithm determines if the search string matches the root node (which is m). Because "c" comes before "m", the entire right subtree of the root node is eliminated and therefore immediately eliminates half of the potential search space.

The intention behind creating a BST is to serve as an in-memory cache created from individual member records retrieved from a database and utilized to eliminate redundant database queries associated with high frequency lookups. In spite of this purpose, in the initial version of this implementation, the BST is created and

populated on a per-request basis. Although this approach guarantees that every record contained in the tree will be current, there will also be an $O(n)$ cost associated with fetching each record from the database prior to executing each $O(\log n)$ search operation. Making this structure an application wide singleton is an area that is targeted for improvement in terms of optimizing system performance.

Chapter 3

Implementation and Analysis

A lot of the system's integrity relies upon two central areas of focus: managing the custom algorithmic data structures that control the mapping and searching of member data during run time, and enforcing user authentication through cryptography in order to protect member credentials from unauthorized access by others.

3.1 Operations with the Custom Binary Search Tree

The custom BST dictates how member data is both mapped and searched at run time. Because binary search trees are index-free (unlike standard arrays), they require comparisons to determine which direction to navigate at each intersection of nodes.

Algorithm 1: Binary Search Tree Node Insertion Algorithm 1 inserts new members into the tree after the members have been either registered or retrieved from a source. Algorithm 1 performs recursive traversal of the tree until the location for the new member has been identified. Algorithm 1 ensures that any node to the left of the current node will always have an email address that is alphabetically less than the current node, while any node to the right of the current node will always have an email address that is greater than the current node. This creates a consistent, non-varying relationship that supports the structural properties of the tree.

```

ALGORITHM InsertMemberNode(RootNode , NewMember)
    IF RootNode is NULL THEN
        RETURN Create_New_Node(NewMember)
    END IF

    // Logically determine the traversal path
    IF NewMember.Email < RootNode.Email THEN
        RootNode.Left = InsertMemberNode(RootNode.Left , NewMember)
    ELSE IF NewMember.Email > RootNode.Email THEN
        RootNode.Right = InsertMemberNode(RootNode.Right , NewMember)
    ELSE
        RAISE DuplicateEmailException
    END IF

    RETURN RootNode
END ALGORITHM

```

Figure 3.1: BST Node Insertion

Algorithm 2: Case Insensitive Searching of Members Case insensitive searches (i.e. “JOHN” = “john”) are required for ensuring standardized formatting of user input regardless of whether users use capitalization or lower case letters when entering names for searching. Under standard ASCII/Unicode ordering rules (“Z” > “a”), without converting all user inputs to lower-case prior to entering the search criteria into the BST, the algorithm will enter incorrect branches and will never identify any existing entries.

```

ALGORITHM SearchMemberByEmail(RootNode , TargetSearchEmail)
    Target = TO_LOWER(TargetSearchEmail)
    CurrentNode = RootNode

    WHILE CurrentNode is NOT NULL DO
        CurrentEmail = TO_LOWER(CurrentNode.Email)

        IF Target == CurrentEmail THEN
            RETURN CurrentNode.MemberData
        ELSE IF Target < CurrentEmail THEN
            // Halves the remaining search space
            CurrentNode = CurrentNode.Left
        ELSE
            // Halves the remaining search space
            CurrentNode = CurrentNode.Right
        END IF
    END WHILE

    RETURN NULL
END ALGORITHM

```

Figure 3.2: Case-Insensitive Member Search

3.2 Secure Authentication System

A widespread misunderstanding about Web Security is the requirement to encrypt passwords. While encryption is reversible (i.e., encrypted information can be decrypted), if an attacker were able to obtain a copy of the AquaSphere database, and subsequently discover the decryption key, he/she could decrypt all stored passwords.

In order to avoid this, AquaSphere deliberately employed an irreversible cryptographic hashing function. AquaSphere utilizes a Key Derivation Function referred to as PBKDF2 (Password-Based Key Derivation Function 2). During registration, when a user supplies a password, it is converted into an irreversible hashed string of characters. This means that no matter what is attempted with respect to reverse engineering or guessing, the hashed password string cannot be used to retrieve the user's original password.

Algorithm 3: Fixed Time Based Password Validation When a user attempts to logon, the system takes the user-supplied password (in its plain text form), hashes it utilizing exactly the same parameters utilized previously, and then compares the resulting temporary hash against the hash that was previously securely saved in the database.

```
ALGORITHM VerifyUserLogin(InputPassword, StoredHash, StoredSalt)
    // 1. Deliberately slow down the process using key-stretching
    DerivedKey = PBKDF2(InputPassword, StoredSalt, Iterations=100000)

    // 2. Perform constant-time comparison to mitigate timing attacks
    MatchResult = CONSTANT_TIME_COMPARE(DerivedKey, StoredHash)

    IF MatchResult is TRUE THEN
        RETURN Authenticated
    ELSE
        RETURN Unauthorized
    END IF
END ALGORITHM
```

Figure 3.3: Constant-Time Password Verification

Role of Iteration(s): The Iterations=100000 parameter is an example of a technique referred to as "Key Stretching". Although modern computers can generate billions of guesses per second in a brute force attack, by mandating that the algorithm perform recursion-based hashing operations on passwords 100,000 times, it adds a microscopic amount of delay. As such, even though there is little apparent difference to the end-user (the login remains almost instantaneously responsive – i.e., in a matter of milliseconds), this adds enough mathematical overhead so as to make brute-force attacks based on stolen databases computationally impossible;

making it statistically improbable for hackers to guess individual passwords within years, let alone decades.



Chapter 4

Results and Discussion

4.1 Testing Approach

In addition to implementing the AquaSphere backend using the MSUnit framework with the AAA testing style, we also employed a robust unit testing approach. Unit testing primarily allowed us to confirm the integrity of the data that is being processed by the application's models, as well as the correctness of our custom binary search tree algorithm, while isolating these two elements from each other along with the SQLite database layer. We tested all aspects of the system through a variety of methods. We tested positive to determine if our standard input would produce the same results every time we run the object. We tested edge cases to see how stable our system was when given null values, overly large function strings, and when attempting to retrieve non-existent queries. These three areas will help us verify that our application does not crash due to a `NullReferenceException`. Finally, we tested structural to ensure that our recursive node structures are able to correctly create left and right pointers between parent and child nodes within the BST logic. In addition, we tested precision to ensure that our financial transactions (which utilize double type for point allocation) are not subject to rounding or truncation errors within our algorithms.

4.2 Table of Test Cases

As part of the unit testing process, we ran a total of 14 deterministic unit tests across four main application domains. Below is a table outlining the different test cases that we ran along with the specific goals that were associated with each test case as defined by the various test suites that we ran.

Table 4.1: Summary of Unit Test Cases and Objectives

Component Tested	Test Scenario / Objective	Test Category	Status
Member Model	Validate accurate assignment of member properties (First-Name, LastName, Email) matching input exactly.	Positive	Passed
Member Model	Ensure the constructor prevents null references and overrides them with default empty strings.	Edge Case	Passed
Member Model	Validate the Primary Key (ID) stores assigned integer identity parameters correctly.	Identity	Passed
BST Node	Verify generic data encapsulation within isolated tree nodes, ensuring children are initially null.	Structural	Passed
BST Node	Validate Left and Right pointer assignments correctly map to distinct child nodes without memory overlap.	Structural	Passed
Search Tree	Verify algorithmic accuracy of recursive insertion and successful retrieval by email identifier.	Algorithmic	Passed
Search Tree	Ensure the search algorithm mathematically normalizes case-insensitive input.	Edge Case	Passed
Search Tree	Verify the search algorithm terminates gracefully (returns null) for queries searching non-existent emails.	Edge Case	Passed
Admin Model	Validate accurate assignment of core administrative privilege properties.	Positive	Passed
Admin Model	Ensure structural flexibility when handling optional/null administrative email definitions.	Edge Case	Passed
Staff Model	Validate accurate mapping of staff identity properties (ID, FullName, Email, Password-Hash).	Positive	Passed
Staff Model	Verify the system correctly stores and retrieves excessively long hyphenated names without character truncation.	Edge Case	Passed
Payment Model	Validate transactional data logic handling and strict pa-	Positive	Passed

4.3 Technical Audit Analysis

Although unit testing confirmed that both our data models and algorithms were functioning properly, further analysis of our architecture during a technical audit revealed a serious performance bottleneck related to the creation/destruction of the BST. As previously discussed in the methodology section, our code base currently creates a full explicit BST from an SQLite data fetch in the middle of executing the search. Although this is done to ensure that stale cache data is removed and fresh data is always returned, this method has introduced a significant $O(n)$ database latency penalty before execution occurs, which defeats the purpose of creating an $O(\log n)$ tree structure locally for each search operation.

Chapter 5

Conclusion

The AquaSphere system successfully provides an extremely secure web based portal that allows members to manage their membership through the use of strong encryption methods with irreversible cryptographic password hashing (PBKDF2), along with normalization of the relational database structure. Additionally, the system has been designed to protect against common cyber threats through its design. Although the current swimmer portal is used by swimmers, the back-end architecture for the application has been validated and configured for Staff and Administration user role specific mapping for future additions. Before expanding from the swimmer portal to full on-line facility management roll-out, it is imperative that we address the inefficiency of using in-memory data caching found within our audit.

Chapter 6

Recommendations / Future Work

The first step in the initial phase of refactoring is to transition all the logic in the current BST init (which fetches from the database) to an application-wide singleton so that it does not loop around getting database results, and so it can establish a true $O(\log(n))$ memory cache. Once the above is completed, the main effort of the development will then immediately switch to implementing the integration of both the Staff and Administrative ecosystems.

All subsequent major releases of the system will implement the full functionality of Workforce Management:

- **Staff Operations Portal:** In addition to the swimmer's ability to use their own self-service tool, the system will also create a secure internal dashboard for staff members to view their dynamic shift rota. They will also have the capability to submit holiday or time off request through the portal; they will also be able to enter facility maintenance ticket information natively on the system. Additionally, an Incident Module will be developed for staff to securely enter on-site accidents which will be documented in real-time.
- **Administrative Management Hub:** The currently inactive admin tier will become the active workforce orchestration component. The administrators will have access to comprehensive dashboards where they can visually arrange staff shift rotas and process incoming holiday requests with conflict resolution options available. Also, reports submitted from staff concerning issues related to facility maintenance and/or on-site accidents will be automatically forwarded to the admins' workspace for third party dispatch tracking and auditing purposes.

In combination with the new robust backend data structures being implemented along with these new front-end workforce tools, AquaSphere will eventually evolve into a fully integrated and end-to-end workforce management

system capable of supporting all needs of today's modern aquatic centers.



Bibliography

- [1] Anderson, R., Kaliski, B. (2000). *RFC 2898: Password-Based Cryptography Specification Version 2.0*. Internet Engineering Task Force (IETF).
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [3] Microsoft Corporation. (2024). *ASP.NET Core Minimal APIs Overview*. Microsoft Learn Documentation.
- [4] SQLite Consortium. (2024). *SQLite Query Optimizer Overview and Indexing*. SQLite Official Documentation.